

Guide 3

Exercicio 1

a)

```
matriz = [0.7 0.2 0.1;  
          0.2 0.3 0.5;  
          0.3 0.3 0.4];  
matrizT = transpose(matriz)
```

```
matrizT = 3x3  
    0.7000    0.2000    0.3000  
    0.2000    0.3000    0.3000  
    0.1000    0.5000    0.4000
```

b)

Pensando no problema como uma sequência de "saltos":

- **Ponto de partida (Dia 1):** Está Sol.
- **Salto 1 (Dia 1 para Dia 2):** Queremos que o Dia 2 seja Sol. Sabendo que hoje (Dia 1) está Sol, a probabilidade de amanhã continuar Sol é 0.7.
- **Salto 2 (Dia 2 para Dia 3):** Queremos que o Dia 3 também seja Sol. Devido à Propriedade de Markov (*o sistema não tem "memória" do passado distante, só quer saber o estado atual para decidir o estado seguinte*), para prever o Dia 3 só nos interessa o que aconteceu no Dia 2; o Dia 1 já não importa. Como no Salto 1 assumimos que o Dia 2 foi Sol, a probabilidade de transitar novamente de Sol para Sol volta a ser 0.7.

Sempre que queremos calcular a probabilidade de uma sequência encadeada de eventos acontecer (queremos o Salto 1 **E** o Salto 2), multiplica-se as probabilidades individuais de cada salto.

A probabilidade de dar Sol no Dia2 sabendo que houve Sol no Dia1 é: 0.7

A probabilidade de dar Sol no Dia3 sabendo que houve Sol no Dia2 é: 0.7

Logo $0.7 * 0.7 = 0.49$ (49%)

c)

Qual a probabilidade de haver sol no sabado e domingo sabendo que dia 1 (quarta-feira) foi chuva?

- **Dia 1 (Quarta-feira):** Chuva (Ponto de partida)
- **Dia 2 (Quinta-feira):** ?
- **Dia 3 (Sexta-feira):** ?
- **Dia 4 (Sábado):** Queremos Sol
- **Dia 5 (Domingo):** Queremos Sol

Ao contrário da alínea (b), onde passamos de um dia para o outro diretamente, aqui temos um "buraco" entre quarta-feira e sábado.

Não podemos simplesmente ler a probabilidade direta na tabela do enunciado, porque o tempo pode ter assumido várias combinações ao longo de quinta e sexta-feira, e todas elas afetam a probabilidade de chegar a sábado com sol.

A solução será criar um **vetor de estado inicial** multiplicados pela matriz de transição.

Assumindo a ordem da matriz, o vetor para o dia 1 será:

```
v1 = [0;      % Sol
      0;      % Nuvens
      1];    % Chuva
```

Para descobrir a probabilidade do tempo no Dia 4 (Sábado), precisamos de dar **3 saltos** a partir do Dia 1 (Qua -> Qui, Qui -> Sex, Sex -> Sáb).

Nas matrizes de transição podemos simplesmente elevar a matriz T (da alínea a) à potência de 3, e multiplicá-la pelo vetor inicial (do dia 1). Isso calcula automaticamente todas as ramificações possíveis de tempo que podem ter acontecido na quinta e sexta.

```
v4 = (matrizT^3) * v1;

% Extrair a probabilidade de o Dia 4 (Sábado) ser de Sol (primeira linha do vetor)
prob_sabado_sol = v4(1);

% Se Sábado é Sol, a probabilidade de Domingo ser Sol é 0.7 (direto da tabela original)
prob_domingo_sol = 0.7;

% A probabilidade final (Sábado ser Sol E Domingo ser Sol)
probabilidade_fim_de_semana = prob_sabado_sol * prob_domingo_sol

probabilidade_fim_de_semana =
0.3003
```

d)

O mês de janeiro tem 31 dias. A média (ou o valor esperado) do número de dias num determinado estado (neste caso, o estado do tempo) é a soma das probabilidades de estar nesse estado em cada um dos 31 dias.

Se pensarmos em vetores de estado:

- **Dia 1:** É Sol. Logo, a probabilidade é 100% Sol. O vetor é $v1 = [1; 0; 0]$.
- **Dia 2:** A probabilidade de cada estado é dada por 1 salto: $T^1 * v1$.
- **Dia 3:** A probabilidade é dada por 2 saltos: $T^2 * v1$.
- ... e assim sucessivamente até ao Dia 31, que será $T^{30} * v1$.

O número total de dias esperado para cada clima será a soma de todos estes 31 vetores.

```

v1 = [1;
      0;
      0];

% Vetor para guardar a soma acumulada
media_diasSol = zeros(3, 1);

for i = 1:31
    prob_dia_atual = (matrizT^(i-1)) * v1;

    media_diasSol = media_diasSol + prob_dia_atual;
end

disp('Media de dias, se o primeiro dia for Sol (Sol, Nuvens, Chuva):');

```

Media de dias, se o primeiro dia for Sol (Sol, Nuvens, Chuva):

```
disp(media_diasSol);
```

```

15.1488
 7.5309
 8.3203

```

e)

```

v1 = [0;
      0;
      1];

% Vetor para guardar a soma acumulada
media_diasChuva = zeros(3, 1);

for i = 1:31
    prob_dia_atual = (matrizT^(i-1)) * v1;

    media_diasChuva = media_diasChuva + prob_dia_atual;
end

disp('Media de dias, se o primeiro dia for Chuva (Sol, Nuvens, Chuva):');

```

Media de dias, se o primeiro dia for Chuva (Sol, Nuvens, Chuva):

```
disp(media_diasChuva);
```

```

13.4539
 7.7004
 9.8457

```

Comparando com a pergunta anterior, o facto do mês começar com chuva, a média de dias de chuva aumenta, a média de dias de sol diminui e a média de dias de núvens sobe ligeiramente.

f)

Na alínea d) e e) calculamos a média de dias se o mês começar com Sol ou Chuva, respetivamente.

- Logo, em média, sabemos os X dias de Sol / Chuva no mês inteiro.
- Sabemos as probabilidades da pessoa ter dor dependendo do tempo.
- O número de dias de dor provocados será

```
% Probabilidades de dor (Sol, Nuvens, Chuva)
pain = [0.1;
        0.3;
        0.5];

% Multiplicar a média de cada clima pela sua respetiva probabilidade de dor, e
somar tudo
painSun_total = sum(media_diasSol .* pain);
painRain_total = sum(media_diasChuva .* pain);

disp('Valor esperado (média) de dias com dor quando o mês começa com:');
```

Valor esperado (média) de dias com dor quando o mês começa com:

```
fprintf('Sol: %.2f dias\n', painSun_total);
```

Sol: 7.93 dias

```
fprintf('Chuva: %.2f dias\n', painRain_total);
```

Chuva: 8.58 dias

Exercicio 2

a)

```
matrizT = [1/3 1/4 0;
           1/3 0.55 1/2;
           1/3 1/5 1/2];
```

b)

Assumindo este sistema:

$$A + B + C = 90$$

$$A = 2(B + C)$$

$$B = C$$

Após resolver: $A = 60$, $B = 15$, $C = 15$

```
vi = [60;  
      15;  
      15];
```

c)

```
class30C = (matrizT^30) * vi
```

```
class30C = 3x1  
16.6667  
44.4444  
28.8889
```

d)

Nas teóricas, as Cadeias de Markov regulares, introduz o conceito de **Distribuição Estacionária** ou estado de equilíbrio a longo prazo. A teoria diz-nos que, após um número suficiente de transições (saltos no tempo), o sistema "esquece" completamente o seu estado inicial.

```
vd = [30;  
      30;  
      30];
```

```
class30D = (matrizT^30) * vd
```

```
class30D = 3x1  
16.6667  
44.4444  
28.8889
```

Exercicio 3

a)

```
matrizT = [0.16 0 0 0.36;  
           0.36 0 0 0.24;  
           0.24 0 0 0.24;
```

```
0.24 1 1 0.16];
```

b)

```
vi = [1;  
      0;  
      0;  
      0];  
prob5transitions = (matrizT^5) * vi
```

```
prob5transitions = 4x1  
0.2103  
0.1824  
0.1624  
0.4448
```

```
prob10transitions = (matrizT^10) * vi
```

```
prob10transitions = 4x1  
0.1979  
0.1821  
0.1583  
0.4616
```

```
prob100transitions = (matrizT^100) * vi
```

```
prob100transitions = 4x1  
0.1979  
0.1821  
0.1583  
0.4617
```

```
prob200transitions = (matrizT^200) * vi
```

```
prob200transitions = 4x1  
0.1979  
0.1821  
0.1583  
0.4617
```

c)

Um sistema fechado regido por uma Cadeia de Markov, os estados não mudam aleatoriamente para sempre. Se deixarmos o sistema correr durante tempo suficiente ($n \rightarrow \infty$), ele atinge um estado de "equilíbrio dinâmico" conhecido como **Distribuição Estacionária** ou **Probabilidades Limite**.

O objetivo da alínea (c) é calcular esse ponto de estabilização exato usando as leis da Álgebra Linear, sem precisar de simular o tempo a passar.

Para encontrar o vetor de probabilidades limite (vamos chamá-lo de π), ele tem de respeitar duas regras matemáticas:

Regra 1: Estabilidade.

Se o sistema já estabilizou, multiplicar a matriz de transição T pelo vetor π vai dar o próprio vetor π .

$$T * \pi = \pi$$

Podemos reorganizar esta equação para resolver o sistema:

$$(T - I) * \pi = 0$$

Onde I é a matriz identidade. No Matlab, podemos gerar a matriz identidade de tamanho 4x4 com a função `eye(4)`.

Regra 2: Conservação da Probabilidade.

A soma de todas as probabilidades dos quatro estados tem de ser obrigatoriamente 100% (ou 1).

$$\pi_A + \pi_B + \pi_C + \pi_D = 1$$

```
% Regra 1
% Matriz A que representará a subtração T - I
A = matrizT - eye(4);
```

```
A = 4x4
    -0.8400         0         0         0.3600
     0.3600    -1.0000         0         0.2400
     0.2400         0    -1.0000         0.2400
     0.2400     1.0000     1.0000    -0.8400
```

```
% Vetor coluna b de zeros, correspondente ao outro lado da igualdade
```

```
b = [0;
     0;
     0;
     0];
```

```
% Vamos forçar a Regra 2
```

```
% Substituímos a última linha da matriz A por 1s
```

```
% (para representar  $\pi_A + \pi_B + \pi_C + \pi_D$ )
```

```
% Pois atualmente se fizermos a soma das colunas darão todas 0
```

```
A(4, :) = [1 1 1 1];
```

```
% Substituímos também o último valor do vetor b por 1
```

```
% (para representar que a soma dá 1).
```

```
b(4) = 1;
```

```
prob_limite = A \ b
```

```
prob_limite = 4x1
    0.1979
    0.1821
    0.1583
```

0.4617

A conclusão final é que o método iterativo estabilizou exatamente para o limite teórico calculado analiticamente. Isto prova que 100 transições já são tempo suficiente para este sistema específico atingir o seu estado estacionário, tornando as iterações seguintes (como a 200) redundantes.

Exercício 4

Se tivermos uma coluna com os valores [2; 3], a soma é 5. Para transformar isto em probabilidades válidas, dividimos cada número pela soma total da coluna, ficando [2/5; 3/5]. Podemos fazer o mesmo para a matriz inteira e dividir pelas somas das suas colunas usando a divisão elemento-a-elemento: $T = M ./ \text{sum}(M)$;

```
M = rand(20,20);  
T = M ./ sum(M)
```

```
T = 20x20  
0.0077    0.0551    0.0151    0.0037    0.0218    0.0546    0.0214    0.0371 ...  
0.0882    0.0504    0.0824    0.0855    0.0671    0.0915    0.0966    0.0793  
0.0065    0.0547    0.0836    0.0364    0.0696    0.0339    0.0816    0.0599  
0.0102    0.0868    0.0019    0.0134    0.0926    0.1034    0.0796    0.0735  
0.0880    0.0123    0.0029    0.0839    0.0084    0.0497    0.0943    0.0176  
0.0877    0.0153    0.0823    0.0755    0.0328    0.0506    0.0834    0.0329  
0.0200    0.0278    0.0081    0.0408    0.0445    0.0570    0.0200    0.0113  
0.0657    0.0804    0.0692    0.0333    0.0632    0.0230    0.1070    0.0002  
0.0407    0.0466    0.0129    0.0699    0.0863    0.0480    0.0074    0.0688  
0.0624    0.0870    0.0825    0.0138    0.0213    0.0554    0.0554    0.0096  
0.0248    0.0847    0.0631    0.0714    0.0617    0.0515    0.0666    0.0999  
0.0292    0.0055    0.0780    0.0333    0.0238    0.0240    0.0110    0.0758  
0.0829    0.0403    0.0210    0.0014    0.0838    0.1041    0.0277    0.0083  
0.0426    0.0299    0.0290    0.0035    0.0094    0.0503    0.0231    0.0194  
0.1005    0.0859    0.0650    0.0794    0.0487    0.0071    0.0678    0.0863  
:  
:
```

a)

```
sum(T)
```

```
ans = 1x20  
1.0000    1.0000    1.0000    1.0000    1.0000    1.0000    1.0000    1.0000 ...
```

b)

```
% Arrancar no estado 1  
vi = zeros(20,1);  
vi(1) = 1;
```



```

transitions2 = (T^2) * vi;
transitions5 = (T^5) * vi;
transitions10 = (T^10) * vi;
transitions100 = (T^100) * vi;

fprintf('2 transições: %.5f%%\n', transitions2(20) * 100)

```

2 transições: 5.59014%

```

fprintf('5 transições: %.5f%%\n', transitions5(20) * 100)

```

5 transições: 5.79064%

```

fprintf('10 transições: %.5f%%\n', transitions10(20) * 100)

```

10 transições: 5.79071%

```

fprintf('100 transições: %.5f%%\n', transitions100(20) * 100)

```

100 transições: 5.79071%

Conclui-se que a partir de 10 transições o sistema já estabilizou.

Exercício 5

A teoria diz-nos que a matriz de transição de uma rede web real (a matriz H) tem dois grandes "defeitos" que quebram as regras das Cadeias de Markov:

- **Dead-ends (Becos sem saída):** São páginas que não contêm qualquer *link* a apontar para fora. O utilizador chega lá e fica "preso", o que faz com que, matematicamente, a soma da coluna seja zero (a matriz deixa de ser estocástica).
- **Spider traps (Armadilhas):** São páginas (ou grupos de páginas) cujos *links* apenas apontam para si próprias. Ao longo do tempo, elas "sugam" toda a probabilidade do sistema, deixando as restantes páginas com um PageRank de zero. A solução genial da Google para resolver o spider trap foi introduzir um fator de amortecimento (geralmente designado por β). Este fator simula a probabilidade de o utilizador se faltar de estar preso e saltar aleatoriamente para qualquer outra página da web.

```

%A B  C D E  F
H = [0 0  0 0 1/3 0; % A
     1 0  0 0 1/3 0; % B
     0 1/2 0 1 0  0; % C
     0 0  1 0 0  0; % D
     0 1/2 0 0 0  0; % E
     0 0  0 0 1/3 0];% F

```

a)

```

% vetor inicial

```

```
% Se 6 páginas têm igual probabilidade inicial,
% o vetor v0 terá o valor de 1/6 nas seis posições.
```

```
vi = [1/6;
      1/6;
      1/6;
      1/6;
      1/6;
      1/6];

it40A = (H^40) * vi
```

```
it40A = 6x1
    0.0000
    0.0000
    0.3333
    0.3333
    0.0000
    0.0000
```

Na iteração 40 o sistema já estabilizou e acontece o esperado. Como os estados C e D geram uma spider trap o sistema ficará preso em C e D.

As paginas com maior "pagerank" são a C e D.

b)

O dead-end é no estado F

A spider trap é criada pelos estados C e D.

c)

```
H(:,6) = 1/6;

it40C = (H^40) * vi
```

```
it40C = 6x1
    0.0000
    0.0000
    0.5000
    0.5000
    0.0000
    0.0000
```

d)

Vamos aplicar a fórmula completa do PageRank usando $\beta = 0.8$.

- No Matlab, constrói-se a Matriz Google (A) com a seguinte fórmula algébrica:

$$A = \beta H + (1 - \beta) \left(\frac{1}{N} \right) J$$

Onde N é o número de páginas (6) e J é uma matriz 6x6 cheia de 1s (função ones(6, 6) no Matlab).

- Usaremos esta nova matriz A para simular novamente as 40 iterações e observar a estabilização do sistema.

```
A = 0.8 * H + (1-0.8)*(1/6)*ones(6,6);
```

```
it40D = (A^40) * vi
```

```
it40D = 6x1
    0.0661
    0.1189
    0.3428
    0.3164
    0.0897
    0.0661
```

e)

```
n = 0;
% Inicializar como vetores
atual = vi;
anterior = zeros(6, 1);

% O ciclo corre enquanto a MAIOR diferença entre os vetores for >= 10^-4
while max(abs(atual - anterior)) >= 1e-4
    n = n + 1;
    anterior = atual;
    atual = A * anterior;
end

disp(['Número de iterações necessárias: ', num2str(n)]);
```

Número de iterações necessárias: 15

```
disp(atual);
```

```
0.0661
0.1190
0.3428
0.3163
0.0898
0.0661
```

Exercício 6

Na teoria das Cadeias de Markov aplicada à Web, o cálculo do PageRank levanta um problema logístico: a internet tem milhares de milhões de páginas. A nível teórico, calcular o estado estacionário através de um sistema de equações exato (como fizemos no Exercício 3c com a divisão à esquerda no Matlab) é matematicamente perfeito. Contudo, na prática (no mundo da Google), inverter matrizes gigantescas é computacionalmente impossível. É por isso que a Google usa o método **iterativo**: simulam o tempo a passar passo-a-passo até os valores estabilizarem.

Neste exercício, como a nossa rede tem apenas 6 páginas, podemos aplicar os dois métodos (iterativo e não-iterativo) para provar que chegam exatamente ao mesmo resultado.

```
%A B C D E F
H = [0 1/2 1/2 0 1/4 0; % A
     1 0 0 0 1/4 0; % B
     0 0 0 1 1/4 0; % C
     0 0 1/2 0 1/4 1; % D
     0 0 0 0 0 0; % E
     0 1/2 0 0 0 0]; % F
```

a)

```
A = 0.85 * H + (1-0.85)*(1/6)*ones(6,6);

vi = [1/6; 1/6; 1/6; 1/6; 1/6; 1/6];

n = 0;
% Inicializar como vetores
atual = vi;
anterior = zeros(6, 1);

% O ciclo corre enquanto a MAIOR diferença entre os vetores for > 0.01
while max(abs(atual - anterior)) > 0.01
    n = n + 1;
    anterior = atual;
    atual = A * anterior;
end

disp(['Número de iterações necessárias: ', num2str(n)]);
```

Número de iterações necessárias: 9

```
disp(atual);
```

```
0.2131
0.2084
0.2205
0.2166
```

0.0250
0.1164

C tem o maior pagerank

E tem o menor pagerank

b)

```
% Regra 1
% Matriz Z que representará a subtração  $T - I$ 
Z = A - eye(6);

% Vetor coluna b de zeros, correspondente ao outro lado da igualdade
b = [0; 0; 0; 0; 0; 0];

% Vamos forçar a Regra 2
% Substituímos a última linha da matriz A por 1s
% (para representar  $\pi_A + \pi_B + \pi_C + \pi_D$ )
% Pois atualmente se fizermos a soma das colunas darão todas 0
Z(6, :) = 1;

% Substituímos também o último valor do vetor b por 1
% (para representar que a soma dá 1).
b(6) = 1;

non_iterative = Z \ b

non_iterative = 6x1
    0.2123
    0.2107
    0.2174
    0.2201
    0.0250
    0.1146
```